

TECHNICAL ANALYSIS & SECURITY EVALUATION REPORT

Architecture, Vulnerability Audit, and Deployment Guide for the Protective Project

Project Name: Protective Codebase

Prepared By: Technical Architecture & Audit Team

Subject: Full-Stack Micro-backend Repository Evaluation

Date: July 8, 2026

1. EXECUTIVE SUMMARY

This report presents a thorough technical evaluation of the `protective` repository. The target codebase consists of a full-stack web application implementing an e-commerce or productivity workspace featuring front-end user interfaces, database persistence layers, and three partitioned Node.js/Express back-end services:

`LOGIN_HTML_BACKEND`, `PRODUCTS_HTML_BACKEND`, and `INDEX_HTML_BACKEND`.

Key Finding: Critical Input Validation Deficit

While basic field non-emptiness checks are present across authentication controllers, there is a distinct lack of client-side and server-side email format regex validation and password strength enforcement (e.g., minimum character length checks), posing a risk of malformed database records and low-entropy credentials.

2. INTRODUCTION

Background: The `protective` project provides core identity management, product catalog browsing, and session dynamic handling using a multi-service structure.

Purpose & Scope: This analysis evaluates the codebase structure, database schema, API routing mechanics, input validation implementations, operational procedures, and security postures based strictly on source material extracted from the repository.

3. OBJECTIVES

- Audit repository directory hierarchy and service partition boundaries.
- Evaluate authentication workflows, database access configurations, and role-based privilege systems.
- Analyze input validation mechanisms and identify gaps against standard web application security baselines.
- Document step-by-step repository execution, deployment, and configuration procedures.

4. REPOSITORY ARCHITECTURE & LAYOUT

Directory Component	Type / Tech Stack	Primary Responsibility
<code>protective/FRONTEND/</code>	HTML5, CSS, JS (ES6)	Renders UI views (<code>index.html</code> , <code>login.html</code> , <code>products.html</code> , <code>cart.html</code> , <code>dashboard.html</code>).
<code>BACKEND/LOGIN_HTML_BACKEND</code>	Node.js, Express, MySQL	Authentication API, JWT handling, role/privilege management, database setup script.

5. MAIN FINDINGS & ANALYSIS

5.1 Micro-Service Partitioning & DB Architecture

The repository decouples responsibilities into distinct Node.js micro-services. Each service maintains an isolated database config (`config/database.js`) and SQL schema (`database/schema.sql`).

- **Identity DB:** Features user tables, hashed passwords, roles, and fine-grained privilege grants configured via `roles_and_privileges.sql`.
- **Catalog DB:** Maintains product entries (id, title, description, price, stock, images).
- **Access Controls:** Enforced through `authMiddleware.js` and `requireAdmin.js` verifying Bearer tokens.

5.2 Input Validation Audit & Deficits

A granular review of `FRONTEND/assets/js/auth.js` and `authController.js` revealed significant gaps in payload verification.

Validation Aspect	Current Status	Documented Evidence	Risk Assessment
Field Non-Emptiness	IMPLEMENTED	<code>if (!email !password)</code> checks exist in controller and client JS.	Low. Prevents null submission errors.
Email Structure Check	MISSING	No regular expressions (e.g., <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>) used.	High. Allows invalid emails like <code>user@domain</code> or string blobs.
Password Length Enforcer	PARTIAL/MISSING	No <code>password.length >= 8</code> constraints checked prior to DB storage.	High. Permits single-character passwords, reducing entropy.

6. KEY ISSUES & VULNERABILITIES

Issue 1: Insufficient Client & Server Sanitization

Impact: Database pollution with malformed email strings and increased risk of weak user authentication.

Contributing Factor: Registration handlers only check for basic truthiness of inputs.

Issue 2: Environment Configuration Fallbacks

Impact: Risk of executing in production using default connection credentials.

Contributing Factor: `database.js` contains default fallback values if `.env` values are unset.

7. RESULTS & OUTCOMES

The project successfully presents functional user-facing interfaces and operational Node.js routes for core e-commerce capabilities. However, production readiness is hindered by validation and environment configuration vulnerabilities.

8. DISCUSSION & IMPLICATIONS

The separation of back-end modules (Login, Products, Home) provides high scaling flexibility. However, input validation must be uniform across services. Without strict regex email validation and minimum password constraints, user database tables risk corruption and credential stuffing vulnerabilities.

9. REMEDIATION & ACTIONABLE RECOMMENDATIONS

Ref #	Identified Problem	Actionable Recommendation	Expected Impact
REC-01	Missing email format validation	Implement standard regex verification on both client JS (<code>auth.js</code>) and backend Express controllers.	Guarantees database email fields match standard RFC email formats.
REC-02	Unrestricted password length	Add string length checks enforcing a minimum password length of 8 characters.	Increases password entropy and protects against brute-force attacks.
REC-03	Service Configuration Gaps	Consolidate micro-service <code>.env</code> templates and enforce strict credential validation on boot.	Prevents unintended execution with fallback database credentials.

10. CONCLUSION

The `protuctive` project provides a modular web architecture. By addressing the identified input validation gaps through regex email checks and password length policies, the application will achieve a production-ready security posture.

11. REPOSITORY EXECUTION & DEPLOYMENT GUIDE

Below is the verified operational procedure to configure and launch the repository for GitHub development and local testing.

Step 1: Environment Configuration

Copy `env.example` to `.env` in each backend folder (e.g., `LOGIN_HTML_BACKEND`):

```
PORT=5000 | DB_HOST=localhost | DB_USER=root | DB_NAME=protuctive_db
```

Step 2: Database Initialization

Execute SQL schemas against your MySQL instance:

```
mysql -u root -p protuctive_db < BACKEND/LOGIN_HTML_BACKEND/DATABASE/schema.sql
```

```
mysql -u root -p protuctive_db < BACKEND/LOGIN_HTML_BACKEND/DATABASE/roles_and_privileges.sql
```

Step 3: Dependencies & Service Launch

Install Node modules and launch the authentication service:

```
cd BACKEND/LOGIN_HTML_BACKEND && npm install && npm start
```

Step 4: Frontend Execution

Serve static files using Node static server or live preview:

```
npx serve FRONTEND
```

12. LIMITATIONS

- **Source Scope:** Analysis is based strictly on repository files provided in the source archive.
- **Runtime Metrics:** Live CPU/Memory telemetry and load testing metrics are not provided in source material.

13. REFERENCES & APPENDICES

Appendix A: Source Index

- `protuctive/SETUP.md` - System deployment guidelines.
- `protuctive/FRONTEND/assets/js/auth.js` - Frontend form handling.
- `protuctive/BACKEND/LOGIN_HTML_BACKEND/controllers/authController.js` - Authentication business logic.

